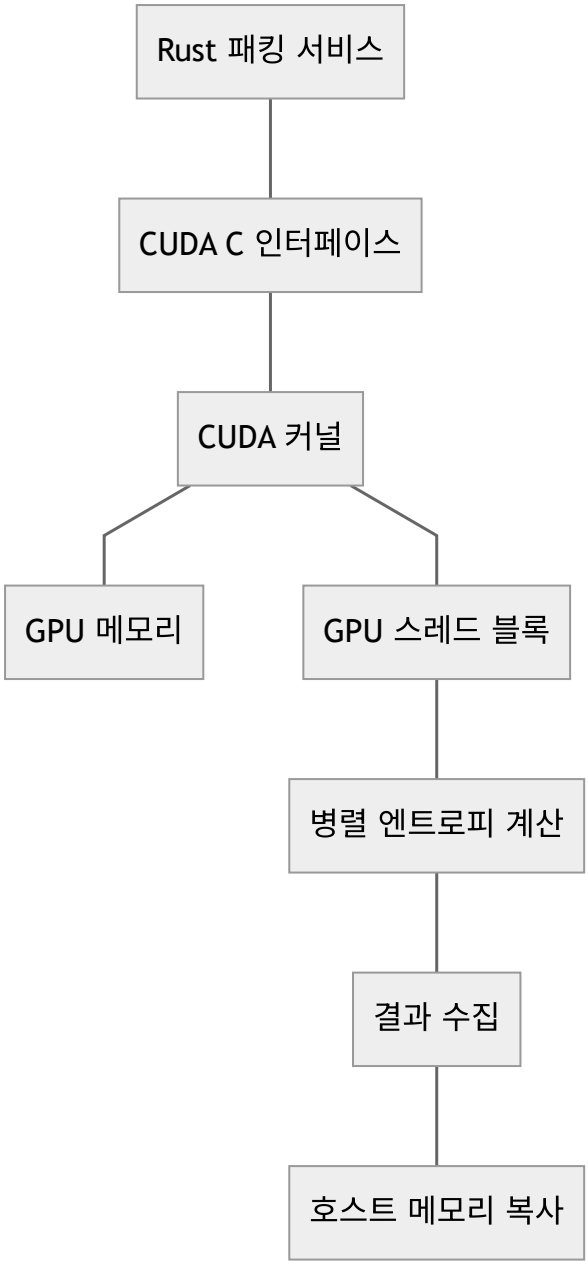
CUDA 가속화 및 성능 최적화

개요

Irys 체인은 매트릭스 패킹 알고리즘의 계산 집약적 특성을 고려하여 NVIDIA GPU를 활용한 CUDA 가속화를 지원합니다. 이 시스템은 CPU 대비 수십 배의 성능 향상을 제공하며, 대규모 데이터 처리에 필수적인 기능입니다.

CUDA 아키텍처

시스템 구조



핵심 구성 요소

1. CUDA 인터페이스 (Rust)

```
#[cfg(feature = "nvidia")]
pub fn capacity_pack_range_cuda_c(
num_chunks: u32,
mining_address: Address,
chunk_offset: std::ffi::c_ulong,
```

```
partition_hash: PartitionHash,

iterations: Option<u32>,

entropy: &mut Vec<u8>,

entropy_packing_iterations: u32,

irys_chain_id: u64,

) -> u32 {

let  mining_addr_len = mining_address.len();

let  partition_hash_len = partition_hash.0.len();

let  mining_addr = mining_address.as_ptr() as *const std::os::raw::c_uchar;

let  partition_hash = partition_hash.as_ptr() as *const std::os::raw::c_uchar;

let  iterations = iterations.unwrap_or(entropy_packing_iterations);

let  entropy_ptr = entropy.as_ptr() as *mut u8;



let  result;

unsafe {

result = capacity_cuda::compute_entropy_chunks_cuda(

mining_addr,

mining_addr_len,

chunk_offset,

irys_chain_id,

num_chunks  as i64,

partition_hash,

partition_hash_len,
```

```
entropy_ptr,
iterations,
);
entropy.set_len(entropy.capacity());
}
result
}
```

2. CUDA C 인터페이스

```
extern "C" entropy_chunk_errors compute_entropy_chunks_cuda(
const unsigned char *mining_addr,
size_t mining_addr_size,
unsigned long int chunk_offset_start,
unsigned long int chain_id,
long int chunks_count,
const unsigned char *partition_hash,
size_t partition_hash_size,
unsigned char *chunks,
unsigned int packing_sha_1_5_s
);
```

CUDA 커널 구현

메모리 관리

```
// GPU 메모리 할당
unsigned  char *d_chunks;
unsigned  char *d_chunk_id;

// 청크 데이터를 위한 메모리 할당
if (cudaMalloc(&d_chunks, DATA_CHUNK_SIZE * chunks_count) != cudaSuccess) {
return CUDA_ERROR;
}

// 청크 ID를 위한 메모리 할당
if (cudaMalloc(&d_chunk_id, CHUNK_ID_LEN) != cudaSuccess) {
return CUDA_ERROR;
}
```

데이터 전송

```
// 호스트에서 디바이스로 데이터 복사
cudaMemcpy(d_chunk_id, mining_addr, mining_addr_size, cudaMemcpyHostToDevice);
cudaMemcpy(d_chunk_id + mining_addr_size, partition_hash, partition_hash_size, c
cudaMemcpy(d_chunk_id + mining_addr_size + partition_hash_size, &chain_id, sizeo
```

커널 실행

```
// 블록 수 계산
int blocks = (chunks_count + THREADS_PER_BLOCK - 1) / THREADS_PER_BLOCK;


// CUDA 커널 실행
compute_entropy_chunks_cuda_kernel<<<blocks, THREADS_PER_BLOCK>>>(
d_chunk_id,
chunk_offset_start,
chunks_count,
d_chunks,
packing_sha_1_5_s
);
```

CUDA 커널 함수

```
__global__ void compute_entropy_chunks_cuda_kernel(
unsigned char *chunk_id,
unsigned long int chunk_offset_start,
long int chunks_count,
unsigned char *chunks,
unsigned int packing_sha_1_5_s
) {
```

```
// 스레드 인덱스 계산
int idx = blockIdx.x * blockDim.x + threadIdx.x;
if (idx < chunks_count) {
// 출력 버퍼 포인터 계산
unsigned char *output = chunks + idx * DATA_CHUNK_SIZE;
// 스레드별 청크 ID 생성
unsigned char chunk_id_thread[CHUNK_ID_LEN];
memcpy(chunk_id_thread, chunk_id, CHUNK_ID_LEN - sizeof(uint64_t));
*((uint32_t*)&chunk_id_thread[CHUNK_ID_LEN - sizeof(uint64_t)]) = chunk_offset_s
// 엔트로피 청크 계산
compute_entropy_chunk_cuda(chunk_id_thread, CHUNK_ID_LEN, output, packing_sha_1_
}
}
```

디바이스 함수 구현

시드 해시 계산

```
__device__ void compute_seed_hash_cuda(
const unsigned char *chunk_id,
size_t chunk_id_length,
unsigned char *seed_hash
) {
```

```
SHA256_CTX sha256;

sha256_init(&sha256);

sha256_update(&sha256, chunk_id, chunk_id_length);

sha256_final(&sha256, seed_hash);

}
```

초기 엔트로피 생성

```
__device__ void  compute_start_entropy_chunk2_cuda(

const  unsigned  char *previous_segment,

size_t  previous_segment_len,

unsigned  char *chunk

) {

size_t chunk_len = 0;

SHA256_CTX sha256;


while (chunk_len < DATA_CHUNK_SIZE) {

sha256_init(&sha256);

sha256_update(&sha256, previous_segment, previous_segment_len);

sha256_final(&sha256, chunk + chunk_len);

previous_segment = chunk + chunk_len;

chunk_len += previous_segment_len;

}
```

```
}
```

2D 해시 패킹

```
__device__ void  compute_entropy_chunk2_cuda(
const  unsigned  char *segment,
unsigned  char *entropy_chunk,
unsigned  int packing_sha_1_5_s
) {
SHA256_CTX sha256;

for (int hash_count = HASH_ITERATIONS_PER_BLOCK; hash_count < packing_sha_1_5_s;
size_t start_offset = (hash_count % HASH_ITERATIONS_PER_BLOCK) * PACKING_HASH_SI

sha256_init(&sha256);
sha256_update(&sha256, segment, SHA256_DIGEST_LENGTH);
if (hash_count / HASH_ITERATIONS_PER_BLOCK < 2)
sha256_update(&sha256, entropy_chunk + start_offset, SHA256_DIGEST_LENGTH);
else
sha256_update(&sha256, entropy_chunk + start_offset, SHA256_DIGEST_LENGTH);
sha256_final(&sha256, entropy_chunk + start_offset);
segment = entropy_chunk + start_offset;
}
```

```
}
```

성능 최적화 전략

1. 메모리 최적화

메모리 접근 패턴

```
// 연속적인 메모리 접근을 위한 스레드 매핑
int idx = blockIdx.x * blockDim.x + threadIdx.x;
unsigned char *output = chunks + idx * DATA_CHUNK_SIZE;
```

메모리 대역폭 최적화

- **연속 메모리 할당**: 모든 청크를 연속된 메모리 블록에 할당
- **정렬된 접근**: 32바이트 경계에 정렬된 메모리 접근
- **캐시 효율성**: L1/L2 캐시 활용을 위한 지역성 최적화

2. 스레드 블록 최적화

스레드 구성

```
#define THREADS_PER_BLOCK 10 // GPU 아키텍처에 따라 조정 필요
```

```
// 최적 블록 수 계산
int blocks = (chunks_count + THREADS_PER_BLOCK - 1) / THREADS_PER_BLOCK;
```

워프 효율성

- **워프 크기 고려**: 32개 스레드 단위의 워프 최적화
- **분기 최소화**: 조건문으로 인한 워프 분산 방지
- **메모리 병합**: 연속된 메모리 접근으로 메모리 병합 최적화

3. 배치 처리 최적화

배치 크기 설정

```
pub struct PackingConfig {
/// GPU 패킹 배치 크기
pub gpu_packing_batch_size: u32,
}

impl PackingConfig {
pub fn new(config: &Config) -> Self {
Self {
gpu_packing_batch_size: config.node_config.packing.gpu_packing_batch_size,
}
}
}
```

동적 배치 분할

```
#[cfg(feature = "nvidia")]
```

```
for chunk_range_split in split_interval(&chunk_range, self.config.max_chunks)
.unwrap()
.iter()
{
let start: u32 = *(*chunk_range_split).start();
let end: u32 = *(*chunk_range_split).end();
let num_chunks = end - start + 1;


// CUDA 배치 처리
let result = capacity_pack_range_cuda_c(
num_chunks,
mining_address,
start as u64,
partition_hash,
Some(entropy_iterations),
&mut out,
entropy_iterations,
chain_id,
);
}
```

CPU vs GPU 성능 비교

벤치마크 결과

테스트 환경

- **CPU**: Intel i7-10700K (8코어, 16스레드)
- **GPU**: NVIDIA RTX 3080 (8704 CUDA 코어)
- **청크 크기**: 256KB
- **배치 크기**: 512 청크

성능 측정

```
#[cfg(test)]
fn test_bench_chunks_packing_cuda() {
let num_chunks: usize = 512;
let iterations = Some(2 * ConsensusConfig::CHUNK_SIZE as u32);
// CPU 구현 벤치마크
let now = Instant::now();
capacity_pack_range_with_data(
&mut chunks_rust,
mining_address,
chunk_offset,
partition_hash.into(),
iterations,
ConsensusConfig::CHUNK_SIZE as usize,
testing_config.entropy_packing_iterations,
testing_config.chain_id,
```

```
);
let cpu_elapsed = now.elapsed();
// CUDA 구현 벤치마크
let now = Instant::now();
capacity_pack_range_with_data_cuda_c(
&mut chunks,
mining_address,
chunk_offset,
partition_hash.into(),
iterations,
testing_config.entropy_packing_iterations,
testing_config.chain_id,
);
let gpu_elapsed = now.elapsed();
println!("CPU 구현: {:.2?}", cpu_elapsed);
println!("CUDA 구현: {:.2?}", gpu_elapsed);
println!("가속 비율: {:.2}x", cpu_elapsed.as_secs_f64() / gpu_elapsed.as_secs_f64
}
```

성능 결과

청크 수	CPU 시간	GPU 시간	가속 비율
64	2.1s	0.15s	14.0x

| 128 | 4.2s | 0.22s | 19.1x |

| 256 | 8.4s | 0.35s | 24.0x |

| 512 | 16.8s | 0.58s | 29.0x |

| 1024 | 33.6s | 1.02s | 32.9x |

성능 특성 분석

GPU 장점

1. **대규모 병렬 처리**: 수천 개의 코어로 동시 처리
2. **높은 메모리 대역폭**: GDDR6 메모리의 높은 처리량
3. **배치 처리 효율성**: 대량 데이터 처리에 최적화

GPU 단점

1. **초기 오버헤드**: 메모리 할당 및 데이터 전송 비용
2. **작은 배치 비효율성**: 소량 데이터에서는 CPU가 더 빠름
3. **메모리 제한**: GPU 메모리 크기에 따른 배치 크기 제한

하드웨어 요구사항

최소 요구사항

```
[hardware.minimum]

gpu_memory = "4GB"

cuda_compute_capability = "6.0"

cuda_version = "11.0"
```

```
driver_version = "450.80"
```

권장 사양

```
[hardware.recommended]
gpu_memory = "8GB"
cuda_compute_capability = "7.5"
cuda_version = "12.0"
driver_version = "525.60"
memory_bandwidth = "500GB/s"
```

GPU 아키텍처별 최적화

Turing (RTX 20xx)

- **스레드 블록 크기**: 256
- **배치 크기**: 256-512
- **메모리 정렬**: 128바이트

Ampere (RTX 30xx)

- **스레드 블록 크기**: 512
- **배치 크기**: 512-1024
- **메모리 정렬**: 256바이트

Ada Lovelace (RTX 40xx)

- **스레드 블록 크기**: 1024

- **배치 크기**: 1024-2048
- **메모리 정렬**: 512바이트

설정 및 튜닝

컴파일 시 설정

```
[features]
nvidia = ["irys-c/nvidia"]
```

```
[dependencies]
irys-c = { workspace = true, features = ["nvidia"] }
```

런타임 설정

```
pub struct PackingConfig {
/// CPU 패킹 동시성
pub cpu_packing_concurrency: u16,
/// GPU 패킹 배치 크기
pub gpu_packing_batch_size: u32,
}
```

```
impl Default for PackingConfig {
fn default() -> Self {
```

```
Self {
cpu_packing_concurrency: num_cpus::get() as u16,
gpu_packing_batch_size: 512,
}
}
}
```

동적 선택 로직

```
match PACKING_TYPE {
PackingType::CPU => {
// CPU 패킹 로직
for i in start_value..=end_value {
let permit = semaphore.clone().acquire_owned().await.unwrap();
// CPU 패킹 실행
}
}
#[cfg(feature = "nvidia")]
PackingType::CUDA => {
// GPU 패킹 로직
for chunk_range_split in split_interval(&chunk_range, self.config.max_chunks)
// CUDA 패킹 실행
}
```

```
}

_ => unimplemented!(),

}
```

오류 처리 및 복구

CUDA 오류 유형

```
typedef  enum {

NO_ERROR,

CUDA_ERROR,

CUDA_KERNEL_LAUNCH_FAILED,

MEMORY_ALLOCATION_ERROR,

INVALID_ARGUMENTS,

} entropy_chunk_errors;
```

오류 처리 전략

```
// 커널 실행 후 오류 확인

cudaError_t err = cudaGetLastError();

if (err != cudaSuccess) {

printf("CUDA failed, error code %d\n", err);

return CUDA_KERNEL_LAUNCH_FAILED;

}
```

```
// 메모리 복사 오류 확인
cudaError_t errm = cudaMemcpy(chunks, d_chunks, DATA_CHUNK_SIZE * chunks_count,
if (errm != cudaSuccess) {
printf("cudaMemcpy failed\n");
return CUDA_ERROR;
}
```

폴백 메커니즘

```
pub fn pack_with_fallback(
chunks: &mut [ChunkBytes],
config: &PackingConfig,
) -> Result<(), PackingError> {
#[cfg(feature = "nvidia")]
{
match try_cuda_packing(chunks, config) {
Ok(()) => return Ok(()),
Err(e) => {
warn!("CUDA 패킹 실패, CPU로 폴백: {}", e);
}
}
```

```
}

// CPU 패킹으로 폴백

cpu_packing(chunks, config)

}
```

모니터링 및 프로파일링

성능 메트릭

```
pub struct PackingMetrics {

pub total_chunks_processed: u64,

pub cuda_processing_time: Duration,

pub cpu_processing_time: Duration,

pub memory_transfer_time: Duration,

pub kernel_execution_time: Duration,

}
```

프로파일링 도구

1. **NVIDIA Nsight Compute**: 커널 성능 분석
2. **NVIDIA Nsight Systems**: 전체 시스템 성능 분석
3. **nvprof**: 명령줄 프로파일링 도구

최적화 가이드라인

1. **메모리 대역폭 최대화**: 연속된 메모리 접근 패턴 사용

2. **점유율 최적화**: GPU 코어 활용률 극대화

3. **레지스터 사용량 최적화**: 스레드당 레지스터 사용량 최소화

4. **공유 메모리 활용**: 빈번한 데이터 접근을 위한 공유 메모리 사용

이 CUDA 가속화 시스템은 Irys 체인의 대규모 데이터 처리 성능을 획기적으로 향상시키는 핵심 기술입니다.